ChatGPT

Dyptgående «ende»-research: end-to-end dyp læring, deep edge-to-end og komplette KI-systemer

Sammendrag

Uttrykket **«dyptgående ende» er ikke en etablert norsk teknologisk fagterm** i de primærkildene som er gjennomgått. Den teknisk mest plausible tolkningen er **end-to-end deep learning**: en modell trenes direkte fra relativt rå input til det ønskede sluttresultatet, slik at mellomrepresentasjoner læres i stedet for å programmeres eksplisitt. Googles forskning på talegjenkjenning og NVIDIAs PilotNet er klassiske eksempler. [1]

En bredere tolkning er **deep end-to-end system**: ikke bare én modell, men hele kjeden fra sensor/data via modell, runtime og maskinvare til beslutning eller handling. En tredje, mindre standardisert tolkning er **deep edge-to-end**: dype modeller fordelt over enhet/edge, eventuelt lokal server og sky. Dagens edge-stakker som PyTorch ExecuTorch, ONNX Runtime, OpenVINO og NVIDIA TensorRT gjør denne tolkningen teknisk relevant. [2]

Den viktigste arkitekturavveiningen er ikke egentlig «end-to-end eller ikke», men **hvor mye som skal optimaliseres samlet, og hvor mye struktur som bør beholdes eksplisitt**. End-to-end kan redusere håndlagde mellomtrinn og optimalisere direkte mot sluttmålet; modulære eller hybride systemer gir ofte bedre diagnostiserbarhet, utskiftbarhet og kontroll. NVIDIAs kjøremodell demonstrerte direkte piksel-til-styring, mens Googles talesystemer viser hvordan CTC, RNN-T, attention og Conformer kan bygge ende-til-ende-modeller med ulike kompromisser mellom kvalitet og strømbar latens. [3]

For edge-systemer er **modellkomprimering, kvantisering, kompilering og maskinvaretilpasning** like viktige som modellarkitekturen. NVIDIA viser for eksempel at lavere presisjon kan gi betydelig bedre TensorRT-ytelse, men understreker samtidig at ytelsesforsøk med kvantisering ikke kan betraktes som produksjonsklare før nøyaktigheten er revalidert. [4]

Det finnes ingen meningsfull universell pris, latens eller energikurve for «end-to-end KI». Disse avhenger av datasett, modell, presisjon, batchstørrelse, runtime, maskinvare og nettverk. Derfor bør et prosjekt styres etter et **målehierarki**: oppgavekvalitet → robusthet/sikkerhet → ende-til-ende-latens → throughput → minne → joule per inferens → kostnad per nyttig prediksjon. MLPerf Inference er et relevant standardisert utgangspunkt for sammenlignbar ytelses- og effektmåling. [5]

Det vedlagte dokumentet beskriver et separat forskningsopplegg for norsk HR, arbeidsliv, syntetiske HR-data og personalhåndbok, og definerer ikke uttrykket «dyptgående ende». Jeg behandler derfor vedlegget som **sidekontekst**, ikke som teknisk kilde for begrepet i denne rapporten.
filecite turn0file0

Begrepsavklaring og kandidatfortolkninger

Det engelske begrepet som har den tydeligste faglige parallellen, er **end-to-end learning**. I denne betydningen lærer systemet en funksjon

$$f_\theta : X \to Y$$

direkte fra treningspar (x, y), og parameterne θ optimaliseres etter en slutt-tapsfunksjon. «Direkte» betyr ikke nødvendigvis at modellen bokstavelig talt bare består av ett lag eller én komponent; den kan inneholde encoder, decoder, attention, konvolusjoner, Transformer-blokker og hjelpehoder. Poenget er at komponentene kan optimaliseres samlet mot sluttoppgaven. Googles sekvens-til-sekvens-arbeid beskriver dette som en generell tilnærming med minimale antakelser om sekvensstrukturen, mens talegjenkjenningsarbeid viser at akustiske, uttalemessige og språklige funksjoner kan læres i ett nevralt system. [6]

I tradisjonell talegjenkjenning kunne kjeden forenklet fremstilles som **signal → akustisk modell → uttaleleksikon → språkmodell → tekst**. I end-to-end-ASR kan modellen i stedet trenes fra lyd til grafemer, subord eller tekstsekvens. Googles sammenligning omfatter blant annet CTC, RNN-Transducer og attention-baserte sekvensmodeller. [7]

I autonom kjøring illustrerte NVIDIA en enda mer konkret variant: et CNN-system ble trent fra kameraets rå piksler til styrekommandoer, i stedet for at utviklere eksplisitt måtte kode separate moduler for blant annet veikanter, kjørefelt og kontroll. Det opprinnelige arbeidet rapporterte kjøring med 30 bilder per sekund på NVIDIA DRIVE PX. Det er et forskningsdemonstrator-eksempel på end-to-end-kontroll, ikke i seg selv dokumentasjon på at en slik arkitektur tilfredsstiller kravene til et komplett sikkerhetskritisk autonomt kjøresystem. [8]

Sammenligning av plausible betydninger

Kandidatfortolkning	Hva «ende» betyr	Hva som trenes/ optimaliseres	Typisk plassering	Typiske algoritmer	Styrke	H
End-to-end deep learning	Fra oppgave-input til slutt-output	Én sammenhengende differensierbar modell eller modellkjede	Sky, datasenter eller edge	CNN, LSTM/ RNN, Transformer, Conformer, CTC, RNN-T	Direkte optimalisering mot sluttmålet	F fo b
Deep end-to-end system	Hele tekniske kjeden fra datakilde til beslutning	Modell + preprocessing + runtime + eventuell kontroll	Hele systemet	Samme som over, pluss MLOps/ runtime	Systemmålinger samsvarer med faktisk brukeropplevelse	« b m
Deep edge-to-end	Fra edge-enhet gjennom eventuell sky til sluttjeneste	Modell(er), partisjonering og inferensplassering	Sensor → edge → gateway → sky	Kvantisert DNN, split inference, on-device inference	Lavere nettverksavhengighet og mulig lokal behandling	R fl v

Kandidatfortolkning	Hva «ende» betyr	Hva som trenes/ optimaliseres	Typisk plassering	Typiske algoritmer	Styrke	H
End-to-end ML/ MLOps	Fra datafangst til overvåket produksjon	Ikke bare modellen; hele livssyklusen	Utvikling + produksjon	Training, export, compile, deploy, monitoring	Reproduserbarhet og operasjonalisering	E p ik m
Hybrid end-to-end	Sluttmålet optimeres samlet, men med strukturerte mellomledd	Hovedmodell + hjelpeoppgaver eller to-pass	Edge og/ eller sky	Multitask, auxiliary losses, two-pass decoding	Kombinerer struktur og global optimalisering	M tr d
Domene-spesifikt «deep end»	Et lokalt produkt-/ prosjektnavn eller metafor	Ubestemt	Ubestemt	Ubestemt	Kan ha presis lokal betydning	I d u

Den viktigste språklige konklusjonen er derfor at jeg **ikke ville brukt «dyptgående ende» som norsk fagterm i en arkitektur- eller kravspesifikasjon**. Mer presise betegnelser er «ende-til-ende dyp læring», «ende-til-ende KI-system», «edge-til-sky KI» eller «ende-til-ende ML-pipeline», avhengig av hva man faktisk mener.

Arkitekturer og algoritmer

Sekvens-til-sekvens-arkitektur. En encoder omdanner inngangen til en latent representasjon, og en decoder produserer utsekvensen. Den klassiske LSTM-baserte sekvens-til-sekvens-modellen viste at en dyp modell kunne lære en direkte sekvenskartlegging uten at oversettelsesprosessen måtte dekomponeres etter klassisk statistisk maskinoversettelse. [12]

CTC og RNN-T. For tale og andre tidsserier er problemet at input- og outputsekvensene ikke har samme lengde eller kjent én-til-én-justering. CTC håndterer latent alignment, mens RNN-T kombinerer encoder og prediksjonsdel slik at strømbar transkripsjon kan gjøres ende-til-ende. Google beskriver RNN-T som en streaming, all-neural sekvens-til-sekvens-arkitektur og viste bruk av wordpieces for bedre sekvensmodellering. [7]

Attention og Transformer/Conformer. Attention gjør at modellen dynamisk kan vekte relevante deler av inngangssekvensen. Conformer kombinerer konvolusjoners modellering av lokale mønstre med Transformers globale interaksjoner. I det opprinnelige Conformer-arbeidet rapporterte Google 2,1/4,3 prosent WER på LibriSpeech clean/other uten ekstern språkmodell og 1,9/3,9 prosent med ekstern språkmodell; disse tallene er benchmarkresultater for den konkrete forskningsoppskriften, ikke generelle ytelsesgarantier. [13]

Perception-to-control. I robotikk og autonome systemer kan modellen lære

$$\text{sensorer} \rightarrow \text{latent representasjon} \rightarrow \text{handling}$$

i ett treningsoppsett. PilotNet brukte et CNN til direkte kartlegging fra kamerabilder til styring og lærte relevante visuelle trekk uten at hvert slikt trekk var eksplisitt annotert. En senere analyse av modellen viste at nettverket reagerte på blant annet veikanter, kjørefelt og kjøretøy. [8]

Multitask og hjelpehoder. «End-to-end» trenger ikke bety at alle interpretable mellomresultater fjernes. En modell kan trenes med en hovedloss for sluttoppgaven og hjelpelosser for for eksempel dybde, segmentering, objektklasse eller tilstand. Dette gir en mellomposisjon mellom fullstendig monolittisk og klassisk modulær design. Forskningsarbeid innen autonom kjøring har blant annet kombinert semantisk segmentering, dybdekart og kontroll i samme treningsramme. [14]

To-pass/hybrid arkitektur. Googles talegjenkjenning viser hvorfor den rene dikotomien «modulær kontra end-to-end» er for enkel. Et strømmbart RNN-T kan levere første resultat med lav latens, mens en kraftigere ikke-strømbar modell rescår hypotesen. I et rapportert oppsett ga dette 17–22 prosent relativ reduksjon i WER sammenlignet med RNN-T alene, mot en mindre økning i latens. [15]

Edge-to-end-arkitektur. Et praktisk edge-system kan deles slik:

$$\text{sensor} \rightarrow \text{lokal preprocessing} \rightarrow \text{edge-modell} \rightarrow \begin{cases} \text{lokal handling} \\ \text{sky/gateway for tyngre behandling} \end{cases} \rightarrow \text{telemetri} \rightarrow \text{retrainin}$$

PyTorch ExecuTorch er eksplisitt rettet mot on-device inference på alt fra mobiltelefoner til embedded-enheter og mikrokontrollere, mens ONNX Runtime støtter server, edge, mobil og nettleser fra en felles runtime-tilnærming. ExecuTorch kan bruke blant annet CPU-, NPU- og DSP-baserte backends. [16]

En praktisk tommelfingerregel er derfor å velge **full end-to-end** når sluttmålet er veldefinert, treningsdata dekker variasjonen godt og det finnes en sikker måte å validere feil på; **hybrid** når man trenger interpretable eller testbare mellomtilstander; og **modulær** når sikkerhet, feilisolasjon, komponentutskiftning eller regelbaserte garantier veier tyngre enn maksimal felles optimering. Dette er en arkitekturvurdering, ikke en universell regel.

Rammeverk, runtime, maskinvare og kostnader

Dagens stakk skiller stadig tydeligere mellom **modellutvikling** og **produksjonsruntime**. Modellen kan trenes i PyTorch og eksporteres/kompileres til et lettere format eller runtime for edge. ONNX Runtime er laget for å kjøre modeller fra blant annet PyTorch og TensorFlow på flere maskinvareplattformer, mens TensorRT er tett optimalisert mot NVIDIA-maskinvare. [17]

Ledende komponenter og primærkilder

Teknologi	Type	Åpen/ leverandørstyrt	Typisk rolle	Edge	Datasenter	Viktigst egensk
PyTorch / ExecuTorch	Modellrammeverk + edge-runtime	Open source / PyTorch Foundation	Trening, eksport og on-device inferens	Svært relevant	Ja	Samme PyTorch økosys modell enhet; NPU/D backen

Teknologi	Type	Åpen/ leverandørstyrt	Typisk rolle	Edge	Datasenter	Viktigst egensk
ONNX Runtime	Inferensruntime	Open source / Microsoft-økosystem	Portabel inferens mellom modellrammeverk og maskinvare	Ja	Ja	Felles A hardwa executi provide mobilst
OpenVINO	Inferens- og optimaliseringsstakk	Intel-prosjekt	Intel CPU/GPU/ NPU og edge	Ja	Ja	Optima og kjør blant a vision- genera modell Intel-m
NVIDIA TensorRT	Optimaliserende inferensruntime/ compiler	NVIDIA	Optimalisert NVIDIA-inferens	Ja	Ja	Kvantis kernel- optima og ytelses
MLPerf Inference	Benchmark, ikke runtime	MLCommons	Sammenlignbar inferens- og effektmåling	Ja	Ja	Standa modell dataset scenari power submis
Jetson Orin	Edge-maskinvare	NVIDIA	Robotikk, kamera/ vision, lokal inferens	Ja	Nei, primært edge	Integre GPU, m og AI- akseler innen avgren effektb
GPU-datasenterklasse	Maskinvare	Flere leverandører	Trening og tung inferens	Normalt nei	Ja	Stor mi compu kapasit batchin

ExecuTorch fikk i april 2026 status som del av PyTorch Core, etter å ha startet hos Meta som runtime for on-device KI på blant annet mobil, AR/VR, briller og embedded-maskinvare. PyTorch oppgir også at ExecuTorch allerede brukes i ulike AR/VR- og «Family of Apps»-opplevelser hos Meta. [18]

Maskinvarekrav

For **trening** er de kritiske dimensjonene GPU/NPU-regnekraft, akseleratorminne, minnebåndbredde og lagrings-/datalasterytelse. Modellens parameterantall alene er ikke nok til å beregne minnebehov, fordi trening i tillegg krever aktiveringer, gradienter, optimizer-state og ofte flere kopier eller presisjonsformater.

For **edge-inferens** er RAM, minnebåndbredde, termisk effektbudsjett, accelerator-støtte, sensor-I/O og runtime-kompatibilitet minst like viktige som «TOPS». NVIDIA oppga for den opprinnelige Orin-generasjonen blant annet 10–25 W og opptil 100 sparse INT8 TOPS for Orin NX 16 GB, 15–40 W og 200 sparse TOPS for AGX Orin 32 GB samt 15–60 W og 275 sparse TOPS for AGX Orin 64 GB. Dette er teoretiske maskinvarespesifikasjoner; TOPS kan ikke leses som antall faktiske applikasjonsinferenser per sekund. [23]

Kostnadsestimering

Uten modell, datamengde, domene eller SLA er et enkelt budsjettall metodisk svakt. Et mer robust estimat brytes opp slik:

$$C_{\text{total}} = C_{\text{data}} + C_{\text{training}} + C_{\text{hardware/cloud}} + C_{\text{engineering}} + C_{\text{validation}} + C_{\text{operations}}$$

Der:

$$C_{\text{training}} = N_{\text{accelerators}} \times T_{\text{hours}} \times P_{\text{per accelerator-hour}}$$

og for produksjon:

$$C_{\text{inference}} \approx \frac{N_{\text{requests}} \times t_{\text{compute}}}{3600 \times U} \times P_{\text{hour}}$$

hvor U er effektiv utnyttelsesgrad.

Et rent **illustrativt** scenarie med 8 akseleratorer i 500 timer gir 4 000 akseleratortimer. Ved en *antatt* intern eller skyleiepris på 2–8 USD per akseleratortime blir compute-delen 8 000–32 000 USD. Dette er ikke en markedsprisprognose for oktober 2026, men en sensitivitetsberegning som viser hvorfor GPU-timer og utnyttelsesgrad bør behandles som eksplisitte parametere.

Som historisk maskinvareanker publiserte NVIDIA ved Orin-introduksjonen 1KU+-priser på omtrent 599 USD for Orin NX 16 GB, 899 USD for AGX Orin 32 GB og 1 599 USD for AGX Orin 64 GB. Disse tallene var leverandørens volumreferanser fra lanseringsperioden og skal **ikke** brukes som gjeldende innkjøpspris i 2026. [23]

Det mest robuste kostnadsmålet i produksjon er derfor ofte ikke «pris per GPU», men **kostnad per korrekt/nyttig inferens innen SLA**, fordi en billigere node som krever flere noder eller gir for lav kvalitet ikke nødvendigvis er billigst systemmessig.

Data, trening, utrulling og evaluering

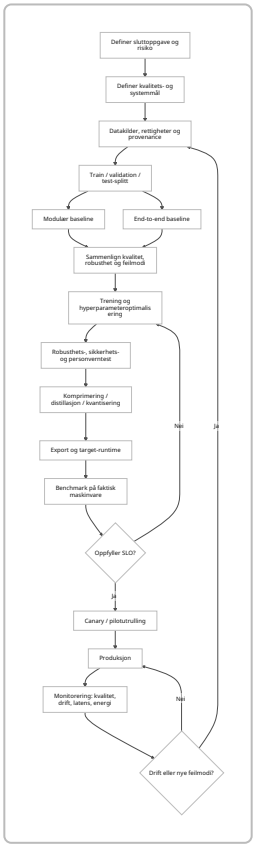
End-to-end-modeller flytter mye av systemdesignen fra håndskrevne regler til **datafordelingen**. PilotNet brukte bilder koblet til menneskelige styrekommandoer; end-to-end talegjenkjenning bruker parallelle lyd–tekst-par. Googles ASR-arbeid viser samtidig en viktig begrensning: fordi en fullstendig

end-to-end-modell i utgangspunktet lærer språkkomponenten fra transkribert lyd, kan tekstmengden være mer begrenset enn i systemer som også utnytter store tekstkorpus. Senere arbeid legger derfor inn eksterne språkdata eller andre hjelpekomponenter. [24]

Dette gir et viktig designprinsipp: **«end-to-end» er et optimeringsvalg, ikke et krav om å forkaste alle eksterne datakilder eller hjelpekomponenter.** Googles RNN-T-arbeid benyttet eksempelvis ekstra tekst-/uttaledata for initialisering, mens two-pass-systemer kombinerer strømbar og ikke-strømbar modell. [25]

Dataene bør dekke både normaltilfeller og de sjeldne hendelsene som dominerer risikoen. For et edge-kamera kan det innebære variasjon i lys, sensor, vinkel, vær og objektfordeling; for tale inkluderer det blant annet støy, dialekt, ordforråd og talestil. Den konkrete dekningsmatrisen må defineres fra domenet, ikke kopieres fra et generelt datasett.

Anbefalt utviklings- og produksjonsflyt



Pipeline-designet reflekterer at modellkomprimering ikke kan valideres bare med runtime-hastighet. NVIDIAs TensorRT-dokumentasjon viser for eksempel et FP8-eksperiment der medianlatensen for den konkrete ViT-testen falt fra omtrent 54,18 til 31,11 ms, men dokumentasjonen sier eksplisitt at resultatet ikke var accuracy-validert og må revalideres med representative data før produksjon. [26]

Evalueringshierarki

En moden evaluering bør separere minst fire målelag:

Lag	Eksempler på mål	Hvorfor
Oppgave	Accuracy, precision/recall/F1, mAP, WER, BLEU, kontrollfeil	Måler om modellen løser oppgaven
Robusthet	Resultat per undergruppe/scenario, kalibrering, feil under distribusjonsskift	Et gjennomsnitt kan skjule alvorlige lommer av svak ytelse
System	p50/p95/p99-latens, throughput, RAM/VRAM, modellstørrelse, cold start	Avgjør om systemet fungerer under faktisk last
Ressurs	Watt, joule/inferens, accelerator-utilization, dataoverføring, kostnad/inferens	Avgjør skalerbarhet og økonomi

For tale er **WER** et sentralt oppgavemål, og Googles forskning bruker det eksplisitt til å sammenligne end-to-end-varianter. For produksjonsruntime anbefaler NVIDIA eksplisitt å lese både throughput og latens i benchmarkingen; TensorRT rapporterer også H2D-/D2H-overføring, compute time og enqueue time separat. [27]

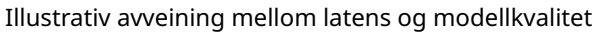
Det er spesielt viktig å måle **ende-til-ende-latens**, ikke bare kernel- eller modelltid:

$$L_{\text{E2E}} = L_{\text{capture}} + L_{\text{preprocess}} + L_{\text{transfer}} + L_{\text{inference}} + L_{\text{postprocess}} + L_{\text{network}} + L_{\text{queue}}$$

TensorRT-dokumentasjonen illustrerer samme poeng ved å skille dataoverføring, GPU-compute og host-køtid i ytelsesrapporteringen. [4]

Ytelse, latens, energi, skalerbarhet og sikkerhet

Latens mot nøyaktighet. Større eller flerpass-modeller kan øke kvaliteten, men bruker typisk mer regning eller venter på mer kontekst. Googles two-pass ASR er et konkret eksempel der et andre pass reduserte WER relativt til RNN-T alene samtidig som noe latens ble lagt til. Den faktiske Pareto-kurven er imidlertid applikasjonsspesifikk. [15]



Illustrativ avveining mellom latens og modellkvalitet

Figuren over er **bevisst skjematisk og ikke et benchmark**. Punktene illustrerer beslutningsformen: det interessante er å finne modeller på Pareto-fronten der ingen annen løsning både er mer nøyaktig og raskere. De numeriske kvalitetstallene i figuren er antakelser, ikke forskningsresultater.

Throughput mot latens. Batching og økt samtidighet kan gjøre maskinvaren bedre utnyttet, men kan også øke ventetiden per forespørsel. NVIDIAs offentlige NIM-målinger viser at både latency og throughput endrer seg med concurrency, og at TensorRT og ONNX kan gi forskjellige resultater på samme GPU-/modelloppsett. Tallene er arbeidslastspesifikke og bør ikke generaliseres til andre modeller. [28]

Energi mot kapasitet. Peak TOPS er en maskinvareindikator, ikke et mål på nyttig applikasjonsgjennomstrømning, og watt er effekt, ikke energi per prediksjon. For energiregnskap er et bedre mål:

$$E_{\text{inference}} = \frac{P_{\text{average}}}{Q_{\text{inferences/sec}}} \quad [J/\text{inference}]$$

MLPerf har egne power submissions nettopp fordi effekt må måles sammen med den faktiske benchmarkarbeidslasten. [5]

Jetson Orin: effekt mot teoretisk AI-kapasitet

Figuren bruker NVIDIAs oppgitte maksimale effektmodi og sparse INT8-TOPS for Orin-generasjonen som **maskinvareproxy**, ikke som faktisk throughput eller joule/inferens. Den illustrerer samtidig hvorfor «275 TOPS» alene ikke er nok som kjøpskriterium. [23]

Skalerbarhet. Cloud/datacenter kan skaleres ved flere akseleratorer og bedre batching, mens edge skaleres ved å distribuere regnekraft ut i hver enhet. Edge kan redusere behovet for å sende alle rådata over nettverket og kan fortsette å levere lokal inferens uten kontinuerlig kontakt med en sentral inferenstjeneste, men krever til gjengjeld versjonsstyring og observabilitet på en potensielt stor flåte. At moderne edge-runtime støtter svært forskjellige maskinvarebackends er synlig i ExecuTorch og ONNX Runtime. [16]

Personvern. Lokal prosessering kan være personvernfremmende når den faktisk gjør det mulig å la rådata bli på enheten, men «on-device» er ikke automatisk det samme som anonymt eller personvernmessig uproblematisk. Datatilsynets norske KI-sandkasse med Ruter illustrerer at formål, behandlingsgrunnlag, åpenhet, informasjon til registrerte og ansvar for lokal lagring må analyseres eksplisitt når personopplysninger inngår i KI-systemet. [29]

Sikkerhet. Et komplett trusselbilde bør minst omfatte manipulert input, kompromittert modellfil, forsyningskjede/runtime, uautorisert modelloppdatering, lekkasje gjennom logger/telemetri, misbruk av API og feil ved rollback. I edge-miljøer kommer også fysisk tilgang og manipulering av selve enheten inn. For industrielle enheter er maskinvaresikkerhet og robusthet derfor en del av arkitekturen; NVIDIA skiller eksempelvis en Industrial Orin-variant med ECC, utvidet temperaturspenn og spesifisert lengre driftsliv fra standardmodulen. [30]

Et særlig end-to-end-problem er **feillokalisering**. Når sluttresultatet er galt, kan årsaken ligge i sensor, distribusjonsskift, latent representasjon, modell, runtime, kvantisering eller outputlogikk. PilotNet-forskernes etterfølgende arbeid med å visualisere hvilke deler av bildet modellen reagerte på illustrerer behovet for diagnostiske mekanismer selv om modellen i utgangspunktet er trent ende-til-ende. [31]

Anvendelser, forskningsspørsmål og anbefalt implementering

Talegjenkjenning er det reneste modenhetseksemplet. Googles forskning viser en utvikling fra CTC/RNN-T/attention-baserte systemer til streaming og Conformer-arkitekturer. End-to-end-systemene kan produsere tekst direkte uten et separat tradisjonelt uttaleleksikon, men forskningen viser også at eksterne tekstdata, rescoring og to-pass-arkitekturer fortsatt kan være verdifulle. Det er derfor feil å forstå teknologihistorien som at «alt modulært ble erstattet av én gigantisk modell». [32]

Autonom kjøring og robotikk er et tydelig forskningscase, men også en advarsel. PilotNet demonstrerte at direkte kamera-til-styring kan læres med et relativt kompakt CNN og menneskelig kjøreatferd som treningssignal. Senere arbeider har kombinert end-to-end-kontroll med semantiske mellomrepresentasjoner og multitask learning, nettopp fordi komplekse kjøresituasjoner gjør en helt ugjennomsiktig mapping vanskelig å analysere og validere. [33]

Edge-KI er nå et eget deployeringslag. ExecuTorch er laget for on-device inferens fra mobil og wearables til embedded og mikrokontrollere, ONNX Runtime dekker både cloud, edge, mobil og nettleser, og Jetson Orin er et eksempel på integrert edge-maskinvare med effektbudsjett i titalls watt. Det gjør lokal tale, vision, robotikk, sensoranalyse og generativ KI teknisk mulig uten at hvert inferenskall må gå til et datasenter. [34]

Norsk relevans. Datatilsynets Ruter-sandkasse er ikke et case på «deep edge-to-end» i snever arkitekturbemerkning, men er relevant som norsk eksempel på at KI-produksjon må koble modellarkitektur til behandlingsgrunnlag, åpenhet, lagringssted og forklaring overfor brukerne. Det er nettopp denne bredere systemforståelsen som ofte mangler når «end-to-end» bare blir behandlet som et modellbegrep. [29]

Åpne forskningsspørsmål

Den største uløste utfordringen er **hvordan man kombinerer global ende-til-ende-optimering med verifiserbare mellomkrav**. En modell kan få bedre sluttmetrikker samtidig som det blir vanskeligere å bevise hvorfor den vil opptre sikkert i sjeldne situasjoner. PilotNet-arbeidet og senere interpretable end-to-end-driving viser denne spenningen tydelig. [35]

Et annet spørsmål er **hvor beregningen skal ligge**. Flere modeller kan flyttes mellom device, lokal edge-server og sky, men optimal partisjonering varierer med nettverkskvalitet, personvern, batteri, temperatur, belastning og modellstørrelse. At ExecuTorch og ONNX Runtime nå dekker stadig flere hardware targets gjør problemet mer løsbart, men også mer flerdimensjonalt. [16]

Kvantisering uten kvalitetsregresjon er fortsatt et praktisk forsknings- og ingeniørproblem. NVIDIAs eget benchmarkeksempel viser hvorfor: FP8 kan gi stor ytelsesgevinst, men leverandøren advarer eksplisitt mot å tolke en performance-test med tilfeldig kalibreringsdata som et ferdig validert produksjonsresultat. [26]

Energi bør flyttes fra maskinvaremarkedsføring til arbeidslastmetrikker. TOPS/W kan være nyttig for en første sortering, men faktisk joule per korrekt inferens krever modell-, runtime- og datasettspesifikk måling. MLPerfs kombinasjon av standardiserte inferensoppgaver og egne power submissions er en bedre metodisk modell. [5]

Anbefalte neste steg

Første beslutning bør være å **låse betydningen av «ende»**. For et konkret prosjekt bør dette uttrykkes som én testbar setning, for eksempel: «kamera-RGB → defektklasse lokalt på edge», «lyd → norsk transkripsjon på mobil», eller «sensorstrøm → kontrollsignal innen 30 ms». Før dette er definert, gir diskusjoner om Transformer kontra CNN eller Jetson kontra sky begrenset verdi.

Deretter bør man bygge **to baselines**, ikke én: en enkel modulær løsning og en end-to-end/hybrid løsning. Dette gjør det mulig å teste om end-to-end faktisk kjøper noe i form av sluttkvalitet, eller bare flytter kompleksiteten fra kode til data og modell.

Målkriteriene bør fryses **før** modellvalget: oppgavekvalitet, p95/p99 ende-til-ende-latens, maksimum RAM/VRAM, effekt eller joule/inferens, robusthetskrav, feilmodus, personvernkrav og kostnad per inferens. TensorRT- og MLPerf-metodikken viser hvorfor throughput alene ikke er tilstrekkelig. [36]

For edge bør den valgte modellen deretter eksporteres og måles **på den faktiske målmaskinvaren**, ikke bare på utvikler-GPU. ExecuTorch, ONNX Runtime, OpenVINO og TensorRT finnes nettopp fordi modellens produksjonsytelse avhenger av runtime og hardware target, ikke bare av treningsrammeverket. [37]

Praktisk implementeringssjekkliste

- [ ] Definer eksakt input, output og hva «end-to-end» skal omfatte.
- [ ] Definer en modulær baseline og en end-to-end/hybrid baseline.
- [ ] Lås train/validation/test og egne sjeldne/risikofylte scenarioer før optimalisering.
- [ ] Mål både oppgavekvalitet og kalibrering/feil per relevant scenario.
- [ ] Sett SLO for p50, p95 og p99 **ende-til-ende**-latens, ikke bare modelltid.
- [ ] Benchmark FP32/FP16/BF16/INT8/FP8 der target-runtime støtter det, og revalider kvalitet etter hver komprimering. NVIDIAs TensorRT-eksempel viser hvorfor revalidering er nødvendig. [26]
- [ ] Mål peak RAM/VRAM, throughput, gjennomsnittseffekt og joule per inferens på faktisk target-hardware; bruk MLPerf-metodikk som referanse der det passer. [5]
- [ ] Gjør et eksplisitt edge-versus-cloud-valg basert på latens, nettverksavhengighet, databehandling, energi og driftsmodell.
- [ ] Dokumenter persondataflyt, formål, lagring, telemetri og modelloppdatering før produksjon; den norske Ruter-sandkassen illustrerer betydningen av å behandle dette som en systemegenskap. [29]
- [ ] Implementer signerte/autoriserte modelloppdateringer, versjonering, rollback og observabilitet.
- [ ] Canary-test ny modell/runtime før full flåteutrulling.
- [ ] Overvåk kvalitetsdrift, inputdrift, p99-latens, ressursbruk og nye feilmodi etter utrulling.
- [ ] Beregn kostnaden som data + trening + inferens/hardware + validering + drift, ikke bare innkjøpspris på akseleratoren.
- [ ] Behold en eksplisitt «stoppregel»: dersom end-to-end-løsningen ikke slår den modulære baselinen på den samlede kvalitets-, sikkerhets- og driftsprofilen, er større modellkompleksitet ikke i seg selv et mål.

Den mest robuste arbeidshypotesen er dermed at **«dyptgående ende» bør tolkes som «ende-til-ende dyp læring» med en mulig utvidelse til edge-til-sky-systemet rundt modellen**. Teknologisk bør det ikke behandles som én bestemt algoritme, men som et designprinsipp: bestem hvilke grenser som skal optimaliseres samlet, behold eksplisitt struktur der den gir sikkerhet eller diagnoserbarhet, og evaluer alltid den komplette kjeden på faktisk målmaskinvare. Googles utvikling fra rene sekvensmodeller til streaming og two-pass ASR, NVIDIAs direkte perception-to-control-forskning og dagens edge-runtime-stakker peker alle i samme retning: den praktisk sterkeste løsningen er ofte **ende-til-ende optimalisert, men ikke nødvendigvis ende-til-ende monolittisk**. [38]

---

[1] [7] [9] [27] [32] [38] A Comparison of Sequence-to-Sequence Models for Speech Recognition
https://research.google/pubs/a-comparison-of-sequence-to-sequence-models-for-speech-recognition/?utm_source=chatgpt.com

[2] [18] ExecuTorch Becomes a Part of PyTorch Core to Expand On-Device Inference Capabilities - PyTorch
https://pytorch.org/blog/executorch-becomes-part-of-pytorch-core/?utm_source=chatgpt.com

[3] [8] [24] [33] End to End Learning for Self-Driving Cars
https://arxiv.org/abs/1604.07316?utm_source=chatgpt.com

4 26 36 Performance Benchmarking — NVIDIA TensorRT
https://docs.nvidia.com/deeplearning/tensorrt/11.4.0/performance/benchmarking.html?utm_source=chatgpt.com

5 MLPerf® Inference Benchmark Suite - MLPerf Inference Documentation
https://docs.mlcommons.org/inference/index_gh/?utm_source=chatgpt.com

6 12 Sequence to Sequence Learning with Neural Networks
https://research.google/pubs/sequence-to-sequence-learning-with-neural-networks/?utm_source=chatgpt.com

10 16 34 37 Executorch - PyTorch
https://pytorch.org/projects/executorch/?utm_source=chatgpt.com

11 15 Two-Pass End-to-End Speech Recognition
https://research.google/pubs/two-pass-end-to-end-speech-recognition/?utm_source=chatgpt.com

13 Conformer: Convolution-augmented Transformer for Speech Recognition
https://research.google/pubs/conformer-convolution-augmented-transformer-for-speech-recognition/?utm_source=chatgpt.com

14 End-to-end Autonomous Driving with Semantic Depth Cloud Mapping and Multi-agent
https://arxiv.org/abs/2204.05513?utm_source=chatgpt.com

17 19 ONNX Runtime | Inference
https://onnxruntime.ai//inference?utm_source=chatgpt.com

20 OpenVINO™ Test Drive — OpenVINO™ documentation
https://docs.openvino.ai/2025/about-openvino/openvino-ecosystem/openvino-project/openvino-test-drive.html?utm_source=chatgpt.com

21 Jetson AGX Orin für Robotik der nächsten Generation | NVIDIA
https://www.nvidia.com/de-de/autonomous-machines/embedded-systems/jetson-orin/?utm_source=chatgpt.com

22 Achieving High Mixtral 8x7B Performance with NVIDIA H100 Tensor Core GPUs and NVIDIA TensorRT-LLM | NVIDIA Technical Blog
https://developer.nvidia.com/blog/achieving-high-mixtral-8x7b-performance-with-nvidia-h100-tensor-core-gpus-and-tensorrt-llm/?utm_source=chatgpt.com

23 Delivering Server-Class Performance at the Edge with NVIDIA Jetson Orin | NVIDIA Technical Blog
https://developer.nvidia.com/blog/delivering-server-class-performance-at-the-edge-with-nvidia-jetson-orin/?utm_source=chatgpt.com

25 Exploring Architectures, Data and Units For Streaming End-to-End Speech Recognition with RNN-Transducer
https://research.google/pubs/exploring-architectures-data-and-units-for-streaming-end-to-end-speech-recognition-with-rnn-transducer/?utm_source=chatgpt.com

28 Performance — NVIDIA NIM for Multimodal Safety
https://docs.nvidia.com/nim/multimodal-safety/latest/performance.html?utm_source=chatgpt.com

29 Ruter, sluttrapport: I rute med kunstig intelligens | Datatilsynet
https://www.datatilsynet.no/regelverk-og-verktoy/sandkasse-for-kunstig-intelligens/ferdige-prosjekter-og-rapporter/ruter-sluttrapport-i-rute-med-kunstig-intelligens/mal-for-sandkasseprosjektet/?utm_source=chatgpt.com

30 Step into the Future of Industrial-Grade Edge AI with NVIDIA Jetson AGX Orin Industrial | NVIDIA Technical Blog
https://developer.nvidia.com/blog/step-into-the-future-of-industrial-grade-edge-ai-with-nvidia-jetson-agx-orin-industrial?utm_source=chatgpt.com

31 35 Explaining How a Deep Neural Network Trained with End-to-End Learning Steers a Car
https://arxiv.org/abs/1704.07911?utm_source=chatgpt.com